

```
if __name__ == '__main__':  
    main() # Call main() if this module is run, but not when imported.
```

Functionally, this program is the same as the non-OOP tic-tac-toe program. The output looks identical. We've moved the code that used to be in `getBlankBoard()` to the `TTTBoard` class's `__init__()` method, because they perform the same task of preparing the board data structure. We converted the other functions into methods, with the `self` parameter replacing the old `board` parameter, because they also serve a similar purpose: they're both blocks of code that operate on a tic-tac-toe board data structure.

When the code in these methods needs to change the dictionary stored in the `_spaces` attribute, the code uses `self._spaces`. When the code in these methods need to call other methods, the calls would also be preceded by `self` and a period. This is similar to how `coinJars.values()` in “Creating a Simple Class: WizCoin” had an object in the `coinJars` variable. In this example, the object that has the method to call is in a `self` variable.

Also, notice that the `_spaces` attribute begins with an underscore, meaning that only code inside the methods of `TTTBoard` should access or modify it. Code outside the class should only be able to modify `_spaces` indirectly by calling methods that modify it.

It can be helpful to compare the source code of the two tic-tac-toe programs. You can compare the code in this book or view a side-by-side comparison at <https://autbor.com/compareoop/>.

Tic-tac-toe is a small program, so it doesn't take much effort to understand. But what if this program were tens of thousands of lines long with hundreds of different functions? A program with a few dozen classes would be easier to understand than a program with several hundred disparate functions. OOP breaks down a complicated program into easier-to-understand chunks.

Designing Classes for the Real World Is Hard

Designing a class, just like designing a paper form, seems deceptively straightforward. Forms and classes are, by their nature, simplifications of the real-world objects they represent. The question is, how should we